

Análise de Código Java

Problema da Mochila (0/1 Knapsack) · Programação Dinâmica

Linguagem: Java	Padrão: Programação Dinâmica Iterativa	Complexidade: $O(n \cdot W)$ tempo e espaço
-----------------	--	---

1. Objetivo Geral

A classe **Knapsack** resolve o clássico **Problema da Mochila 0/1**: dado um conjunto de n itens, cada um com um peso e um valor associado, e uma mochila com capacidade máxima W , o objetivo é selecionar um subconjunto de itens (sem fracionamento) que maximize o valor total sem exceder a capacidade. O método estático **knapsack()** retorna esse valor máximo, e o método **main()** demonstra dois cenários de teste.

2. Lógica e Algoritmo

O algoritmo utiliza **Programação Dinâmica (bottom-up)**. É construída uma tabela **dp[i][w]**, onde cada célula representa o valor máximo alcançável usando os primeiros i itens com capacidade w . Para cada item i e cada capacidade w , há duas opções: **não incluir** o item (herdar $dp[i-1][w]$) ou **incluir** o item se seu peso couber ($dp[i-1][w - \text{peso}] + \text{valor}$). O maior dos dois é armazenado.

```
dp[i][w] = dp[i-1][w]; // não inclui o item i
if (weights[i-1] <= w) dp[i][w] = Math.max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1]);
```

Recorrência central do algoritmo

3. Conceitos Java Utilizados

Conceito	Descrição
Arrays (int[])	Armazenamento de pesos, valores e tabela DP bidimensional
Método estático	knapsack() e main() são static — chamáveis sem instanciar a classe
Math.max()	Seleciona o maior valor entre incluir ou não o item
Array 2D (int[][])	Tabela DP de dimensões $(n+1) \times (\text{capacidade}+1)$
Loop aninhado	Dupla iteração: itens $\times$ capacidades, $O(n \cdot W)$ operações

4. Complexidade

Tempo: $O(n \cdot W)$ — dois laços aninhados percorrem todos os itens e todas as capacidades. **Espaço:** $O(n \cdot W)$ — a tabela dp ocupa $(n+1) \times (W+1)$ células inteiras. Uma otimização possível (não aplicada aqui) é usar apenas dois arrays 1-D ou um único array percorrido de trás para frente, reduzindo o espaço para $O(W)$.

■ 5. Decisões de Projeto e Limitações

A implementação é **iterativa** (evita o risco de *stack overflow* da versão recursiva) e de fácil compreensão didática. A principal limitação é o **consumo de memória $O(n \cdot W)$** : para capacidades ou quantidades de itens muito grandes, o array pode se tornar inviável. Além disso, o método assume que pesos e valores são **inteiros positivos** — pesos fracionários exigiriam o algoritmo guloso da mochila fracionária.

■ 6. Saída Esperada do main()

Chamada	Capacidade	Resultado	Itens Selecionados
knapsack(w, v, 7)	7	9	Itens de peso 3 e 4 (valores 4 + 5)
knapsack(w, v, 10)	10	13	Itens de peso 1, 4 e 5 (valores 1+5+7)